

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет безопасности информационных технологий

Дисциплина:

«Информационная безопасность баз данных»

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №5

«Резервирование БД и восстановление по контрольным точкам»

Выполнили:

Шибает Иван Дмитриевич, студент группы N3352

(подпись)

Малышев Григорий Игоревич, студент группы
N3352

(подпись)

Проверил:

Ярцева Наталия Андреевна

(подпись)

СОДЕРЖАНИЕ

Введение	3
1 Стратегия и расписание резервного копирования	5
1.1 Выбор способа резервного копирования	5
1.2 Расписание	5
1.3 Настройка архивирования WAL	6
1.4 Сценарий резервного копирования	8
2 Контрольная точка и случайные изменения	11
2.1 Создание контрольной точки	11
2.2 Случайные изменения	12
3 Откат к контрольной точке	17
3.1 Процедура восстановления	17
3.2 Проверка результата	19
4 Анализ откаченных изменений	21
4.1 Журнал изменений лабораторной работы № 3	21
4.2 Архив WAL и утилита pg_waldump	21
4.3 Копия на момент перед откатом	23
4.4 Журнал сервера	25
4.5 Сравнение источников	26
5 Защита резервных копий	28
Заключение	29

ВВЕДЕНИЕ

Цель работы — получение навыков резервирования и восстановления базы данных.

Задание включает четыре пункта:

1. создать резервную копию базы данных согласно выбранному расписанию;
2. внести случайные изменения в таблицы базы данных (изменения вносятся до момента создания контрольной точки);
3. продемонстрировать откат к последней контрольной точке, отменяющий изменения из пункта 2;
4. проанализировать возможность просмотра «откаченных» изменений с помощью системы журналирования СУБД, в том числе журнала из лабораторной работы № 3, или средствами системы резервирования.

Формулировку второго пункта мы понимаем так: изменения вносятся после последней контрольной точки и до создания следующей, то есть ни одна контрольная точка их не фиксирует. Поэтому последовательность работы такая: резервная копия, контрольная точка, случайные изменения, откат к этой контрольной точке.

Под контрольной точкой понимается момент, к которому можно вернуть базу данных. Команда SQL «CHECKPOINT» для этого не подходит: она лишь записывает изменённые страницы из памяти на диск и не создаёт состояния, к которому можно откатиться. В PostgreSQL точкой возврата служит именованная точка восстановления, которую создаёт функция «pg_create_restore_point». Откат к ней выполняется восстановлением базовой копии с применением журнала упреждающей записи (WAL) до этой точки.

Работа выполнена на базе данных «workshop» мастерской реставрации музыкальных инструментов из лабораторной работы № 2 с журналом изменений, шифрованием и ролями из лабораторной работы № 3, СУБД — PostgreSQL 14.21.

Все действия воспроизводятся сценарием «check.sh» (команда «make check»). Сценарий создаёт отдельный учебный кластер на порту 55433

в каталоге «work», строит в нём базу данных сценарием лабораторной работы № 3, выполняет все пункты задания и сохраняет выводы в каталог «verification». Все листинги результатов в отчёте получены этим сценарием. Длинный путь к папке работы в выводах заменён на «.../lab5».

1 Стратегия и расписание резервного копирования

1.1 Выбор способа резервного копирования

В PostgreSQL есть два основных способа резервного копирования: логическая копия утилитой «pg_dump» и физическая копия каталога данных утилитой «pg_basebackup» вместе с архивом WAL. Их сравнение приведено в таблице 1.

Таблица 1 — Сравнение способов резервного копирования

Свойство	pg_dump	pg_basebackup + архив WAL
Содержимое	одна БД: схема и данные в виде команд или архива	весь кластер: все БД, роли, настройки
Момент восстановления	только момент создания копии	любой момент после базовой копии (PITR)
Выборочное восстановление	отдельных таблиц	только кластер целиком
Потеря данных при сбое	всё после последней копии	только WAL, ещё не отправленный в архив
Перенос на другую версию СУБД	возможен	невозможен

Задание требует отката к контрольной точке, созданной после резервной копии. С одной логической копией это невозможно: она восстанавливает только момент своего создания. Поэтому основной способ — физическая базовая копия и непрерывное архивирование WAL. Логическая копия создаётся дополнительно: она не зависит от физического формата файлов, позволяет восстановить отдельную таблицу и остаётся полезной, если повреждение попало и в физические копии.

1.2 Расписание

Выбранное расписание приведено в таблице 2.

Таблица 2 — Расписание резервного копирования

Операция	Когда	Хранение
Архивирование сегмента WAL	непрерывно, по заполнении сегмента и не реже раза в 5 минут	от начала самой ранней хранимой базовой копии
Базовая копия, проверка манифеста	ежедневно в 02:30	7 последних копий
pg_dump БД и pg_dumpall ролей	ежедневно в 02:30, в том же задании	вместе с базовой копией

Обоснование выбора:

- кластер учебной базы данных занимает около 51 МБ (листинг 7), поэтому ежедневная полная копия дешёвая. Еженедельная копия увеличила бы время восстановления: пришлось бы применять WAL за неделю;
- при потере сервера с сохранившимся архивом теряются только транзакции из незаполненного сегмента WAL. Параметр «archive_timeout = 300» ограничивает эти потери пятью минутами;
- при логической ошибке, например ошибочном изменении данных, базу можно вернуть на любой момент в пределах семи суток хранения копий;
- время восстановления складывается из копирования базовой копии и применения WAL не более чем за сутки. На стенде все сообщения журнала сервера о восстановлении имеют одну и ту же отметку времени с точностью до секунды (листинг 17).

Расписание оформлено строкой crontab (листинг 1). В систему оно не устанавливалось: сценарий «check.sh» запускает задание «backup.sh» вручную, чтобы проверка была воспроизводимой и не меняла настройки компьютера.

```
# Расписание резервного копирования ЛР5 (crontab -e).
# Пример, в систему не установлен. LAB5 – путь к работе.
LAB5="/path/to/subjects/ibbd/lab5"
# Ежедневно в 02:30: базовая копия, pg_dump, ротация.
30 2 * * * "$LAB5/backup.sh" >>"$LAB5/work/backup.log" 2>&1
# Архивация WAL выполняется сервером непрерывно
# (archive_command), отдельного задания не требует.
```

Листинг 1 — Расписание задания («crontab.example»)

1.3 Настройка архивирования WAL

После создания кластера в «postgresql.conf» добавляются настройки из листинга 2, в которых метка «@LAB5@» заменяется на путь к папке работы. Уровень «wal_level = replica» сохраняет в WAL всё необходимое для восстановления из архива. При «archive_mode = on» сервер вызывает «archive_command» для каждого заполненного сегмента. Префикс строк

журнала сервера дополнен ролью и базой данных; это используется при анализе в разделе 4.

```
# lab5: непрерывное архивирование WAL (@LAB5@ -> путь ЛР5)
wal_level = replica
archive_mode = on
archive_command = '"@LAB5@/archive_wal.sh" "%p" "%f"'
# Незаполненный сегмент архивируется не реже раза в 5 мин.
archive_timeout = 300
# Журнал сервера: время, роль, БД и операторы изменения.
log_line_prefix = '%m [%p] %u@%d '
```

Листинг 2 — Настройки архивирования («conf/lab5.conf.in»)

Команда архивирования вынесена в сценарий «archive_wal.sh» (листинг 3). Путь к работе содержит пробелы и кириллицу, поэтому все пути в сценарии заключены в кавычки. Сценарий не перезаписывает уже существующий файл архива, как рекомендует документация, и копирует сегмент под временным именем, чтобы в архиве не оказался недописанный файл.

```
#!/bin/sh
# archive_command: копирует сегмент WAL в архив.
# Вызов из PostgreSQL: archive_wal.sh %p %f
# Существующий файл не перезаписывается: при повторе
# PostgreSQL получит ошибку и повторит попытку позже.
set -eu
ARCHIVE="${WAL_ARCHIVE:-$(dirname "$0")/work/wal_archive}"
test ! -f "$ARCHIVE/$2"
cp "$1" "$ARCHIVE/$2.tmp"
mv "$ARCHIVE/$2.tmp" "$ARCHIVE/$2"
```

Листинг 3 — Команда архивирования («archive_wal.sh»)

Действующие значения параметров, прочитанные из «pg_settings», приведены в листинге 4. Параметр «log_statement = mod» включается командой «ALTER SYSTEM» только после сборки базы данных, чтобы в журнал сервера не попали пароли ролей и ключ шифрования из скриптов лабораторной работы № 3.

name		setting
-----+-----		

```

archive_command | ".../lab5/archive_wal.sh" "%p" "%f"
archive_mode    | on
archive_timeout | 300
log_statement   | mod
wal_level       | replica
(5 rows)

```

Листинг 4 — Параметры архивирования и журналирования

1.4 Сценарий резервного копирования

Плановое задание «backup.sh» (листинг 5) выполняет:

1. базовую копию «pg_basebackup» в каталог с отметкой времени. Параметр «-Xstream» включает в копию WAL, необходимый для её согласованности, а «-checkpoint=fast» запускает контрольную точку немедленно;
2. проверку копии по манифесту утилитой «pg_verifybackup»: сверяются список файлов, их размеры и контрольные суммы;
3. логическую копию базы данных «pg_dump -Fc» и выгрузку ролей «pg_dumpall -globals-only»;
4. удаление копий сверх «KEEP» последних и очистку архива от сегментов WAL старше самой ранней оставшейся копии утилитой «pg_archivecleanup». Первый нужный сегмент берётся из файла «backup_label» этой копии.

Маска «umask 077» делает копии доступными только владельцу.

```

#!/bin/sh
# Плановое резервное копирование кластера ЛР5 (задание cron).
# 1) физическая базовая копия pg_basebackup и её проверка;
# 2) логическая копия БД workshop (pg_dump) и ролей;
# 3) хранение KEEP последних копий, очистка архива WAL.
set -eu
LAB_DIR=$(cd "$(dirname "$0")" && pwd)
PGHOST=127.0.0.1
PGPORT="${IBBD_PGPORT:-55433}"
PGUSER=postgres
export PGHOST PGPORT PGUSER
ROOT="${BACKUP_ROOT:-$LAB_DIR/work/backups}"

```

```

ARCHIVE="${WAL_ARCHIVE:-$LAB_DIR/work/wal_archive}"
KEEP="${KEEP:-7}"
DB=workshop

umask 077
stamp=$(date +%Y%m%d-%H%M%S)
dest="$R00T/$stamp"
mkdir -p "$R00T"
log() { echo "$(date +%H:%M:%S) $*"; }

log "base backup -> $stamp/data"
pg_basebackup -D "$dest/data" -Fp -Xstream \
  --checkpoint=fast --label="lab5 $stamp"
pg_verifybackup -q "$dest/data"
log "manifest verified"

pg_dump -Fc -f "$dest/$DB.dump" "$DB"
# Роли кластера (с хэшами паролей SCRAM).
pg_dumpall --globals-only -f "$dest/globals.sql"
log "pg_dump: $(pg_restore -l "$dest/$DB.dump" \
  | grep -c 'TABLE DATA') tables"

# Удаление копий сверх KEEР последних.
ls -l "$R00T" | sort -r | tail -n +"$((KEEP + 1))" |
while read -r old; do
  rm -rf "${R00T:?}/$old"
  log "removed old backup $old"
done

# WAL старше самой ранней оставшейся копии не нужен.
oldest=$(ls -l "$R00T" | sort | head -n 1)
first=$(sed -n 's/^START WAL.*(file \(.*\))$/\1/p' \
  "$R00T/$oldest/data/backup_label")
pg_archivecleanup "$ARCHIVE" "$first"
log "WAL archive kept from $first"

```

Листинг 5 — Задание резервного копирования («backup.sh»)

Для демонстрации ротации задание запущено дважды подряд с «KEEP=1», как два последовательных ночных запуска (листинг 6). Второй запуск удалил первую копию и сегмент WAL 00000001000000000000000002, который нужен был только ей.

```

== запуск 1
12:56:11 base backup -> 20260919-125611/data
12:56:12 manifest verified
12:56:12 pg_dump: 11 tables
12:56:12 WAL archive kept from 00000001000000000000000002
== запуск 2
12:56:13 base backup -> 20260919-125613/data
12:56:13 manifest verified
12:56:13 pg_dump: 11 tables
12:56:13 removed old backup 20260919-125611
12:56:13 WAL archive kept from 00000001000000000000000003

```

Листинг 6 — Два запуска задания резервного копирования

Состав копии, права доступа и содержимое архива WAL после второго запуска приведены в листинге 7. Файлы с суффиксом «.backup» — служебные файлы истории копий, которые «pg_archivecleanup» не удаляет. Логическая копия содержит данные 11 таблиц: девяти основных, журнала «change_log» и таблицы секретов «access_secret».

```

== work/backups
51M 20260919-125613/data
56K 20260919-125613/workshop.dump
4.0K 20260919-125613/globals.sql
== права доступа
drwx----- 20260919-125613
drwx----- 20260919-125613/data
-rw----- 20260919-125613/globals.sql
-rw----- 20260919-125613/workshop.dump
== work/wal_archive
00000001000000000000000002.00000028.backup
00000001000000000000000003
00000001000000000000000003.00000028.backup

```

Листинг 7 — Резервная копия и архив WAL

2 Контрольная точка и случайные изменения

2.1 Создание контрольной точки

Сразу после резервного копирования фиксируется состояние базы данных и создаётся точка восстановления. Состояние описывается числом строк и хэшем MD5 от упорядоченного текстового представления всех строк каждой таблицы, включая журнал ЛРЗ (листинг 8). Совпадение хэшей означает совпадение содержимого таблиц.

```
-- Число строк и MD5 упорядоченного содержимого таблиц.
SELECT tab AS table_name, count(*) AS rows,
       md5(string_agg(r, '|' ORDER BY r)) AS md5
FROM (
  SELECT 'customer' tab, x::text r FROM customer x
  UNION ALL SELECT 'instrument_type', x::text
    FROM instrument_type x
  UNION ALL SELECT 'instrument', x::text FROM instrument x
  UNION ALL SELECT 'employee', x::text FROM employee x
  UNION ALL SELECT 'restoration_order', x::text
    FROM restoration_order x
  UNION ALL SELECT 'service', x::text FROM service x
  UNION ALL SELECT 'order_work', x::text FROM order_work x
  UNION ALL SELECT 'inspection', x::text FROM inspection x
  UNION ALL SELECT 'payment', x::text FROM payment x
  UNION ALL SELECT 'change_log', x::text FROM change_log x
) s
GROUP BY tab ORDER BY tab;
```

Листинг 8 — Состояние таблиц («sql/state.sql»)

Состояние до изменений приведено в листинге 9, созданная точка восстановления — в листинге 10. Функция «pg_create_restore_point» записывает в WAL запись с именем точки и возвращает позицию в WAL (LSN) сразу после этой записи.

table_name	rows	md5
change_log	9	1d4d758cd3f6632a996ca20efebbd3f0
customer	9	b579a43ccd8a4c7ff3f910f039edc4e1
employee	8	8629e8b79c995656747f3038f5ff923f
inspection	9	91cae292a9da23533aa8a62e15b32772

instrument		9		c98e72e52aca9192f404215674ccfff9
instrument_type		8		9c2f71742db467466baf3f1b7ae8e1ac
order_work		11		83049cf8bde2264fed123a613fb506e7
payment		8		b64d1b9ebdc1dce18a7991785f17c231
restoration_order		9		42ff20ee4af9edc6ee192546df14a69c
service		8		fbd7283b1c5f41b0c2874a2dd0958b11

(10 rows)

Листинг 9 — Состояние таблиц в контрольной точке

name		lsn		created_at
-----+-----+-----				
lab5_before_changes		0/4000090		2026-09-19 12:56:14

(1 row)

Листинг 10 — Точка восстановления

2.2 Случайные изменения

Изменения вносятся двумя сеансами, чтобы при анализе было видно, кто их выполнил. Строки и значения выбираются функцией «random()», а вызов «setseed» делает выбор воспроизводимым при повторном запуске. Администратор «postgres» меняет телефон заказчика, цены двух услуг, статус работы и удаляет осмотр (листинг 11). Приёмщик «reception_orlova» входит по паролю, добавляет заказчика, переносит срок заказа и регистрирует оплату (листинг 12). Изменения каждого сеанса выполняются в одной транзакции.

```
-- Случайные изменения от имени администратора postgres.
-- setseed делает выбор строк воспроизводимым.
SELECT setseed(0.2026);
BEGIN;
UPDATE customer
SET phone = '+7 999 ' || lpad((random() * 9999999)::int::text,
                             7, '0')
WHERE customer_id = (SELECT customer_id FROM customer
                     ORDER BY random() LIMIT 1)
RETURNING customer_id, phone;

UPDATE service
SET base_price_kopecks = (base_price_kopecks
```

```

        * (1 + random()))::int
WHERE service_id IN (SELECT service_id FROM service
                     ORDER BY random() LIMIT 2)
RETURNING service_id, base_price_kopecks;

UPDATE order_work SET status = 'cancelled'
WHERE (order_id, line_no) = (
    SELECT order_id, line_no FROM order_work
    WHERE status IN ('planned', 'in_progress')
    ORDER BY random() LIMIT 1)
RETURNING order_id, line_no, status;

DELETE FROM inspection
WHERE inspection_id = (SELECT inspection_id FROM inspection
                      ORDER BY random() LIMIT 1)
RETURNING inspection_id, order_id, inspected_on;
COMMIT;

```

Листинг 11 — Изменения администратора («sql/random_admin.sql»)

```

-- Случайные изменения от имени приёмщика reception_orlova.
SELECT setseed(0.5);
BEGIN;
INSERT INTO customer (full_name, phone, email)
VALUES ('Случайный Клиент ' || (random() * 100)::int,
        '+7 900 555-00-00', NULL)
RETURNING customer_id, full_name;

UPDATE restoration_order
SET due_on = due_on + (1 + random() * 30)::int
WHERE order_id = (SELECT order_id FROM restoration_order
                  WHERE status <> 'closed'
                  ORDER BY random() LIMIT 1)
RETURNING order_id, due_on;

INSERT INTO payment (order_id, paid_on, amount_kopecks,
                    method, receipt_no)
SELECT order_id, current_date,
       (1000 + random() * 9000)::int * 100, 'card',
       'RND-' || (random() * 1e6)::int
FROM restoration_order WHERE status <> 'closed'
ORDER BY random() LIMIT 1

```

```
RETURNING payment_id, order_id, amount_kopecks;
COMMIT;
```

Листинг 12 — Изменения приёмщика («sql/random_reception.sql»)

Результаты выполнения приведены в листингах 13 и 14: благодаря «RETURNING» видно, какие строки выбраны случайно и какие значения получены.

```
setseed
-----

(1 row)

BEGIN
customer_id |      phone
-----+-----
          9 | +7 999 0764774
(1 row)

UPDATE 1
service_id | base_price_kopecks
-----+-----
          1 |          914486
          2 |          353834
(2 rows)

UPDATE 2
order_id | line_no | status
-----+-----+-----
          2 |          1 | cancelled
(1 row)

UPDATE 1
inspection_id | order_id | inspected_on
-----+-----+-----
          1 |          1 | 2026-09-01
(1 row)

DELETE 1
COMMIT
```

Листинг 13 — Изменения администратора

```

setseed
-----

(1 row)

BEGIN
customer_id |      full_name
-----+-----
          10 | Случайный Клиент 25
(1 row)

INSERT 0 1
order_id |  due_on
-----+-----
          6 | 2026-09-10
(1 row)

UPDATE 1
payment_id | order_id | amount_kopecks
-----+-----+-----
          11 |          2 |          240500
(1 row)

INSERT 0 1
COMMIT

```

Листинг 14 — Изменения приёмщика

Сравнение состояния после изменений с контрольной точкой приведено в листинге 15. Изменились семь таблиц, в том числе журнал ЛРЗ, в который триггеры добавили восемь записей. Таблицы «employee», «instrument» и «instrument_type» не затронуты.

table	before	after	md5
change_log	9	17	ОТЛИЧАЕТСЯ
customer	9	10	ОТЛИЧАЕТСЯ
employee	8	8	совпадает
inspection	9	8	ОТЛИЧАЕТСЯ
instrument	9	9	совпадает
instrument_type	8	8	совпадает
order_work	11	11	ОТЛИЧАЕТСЯ
payment	8	9	ОТЛИЧАЕТСЯ

restoration_order	9	9	ОТЛИЧАЕТСЯ
service	8	8	ОТЛИЧАЕТСЯ

Листинг 15 — Контрольная точка и состояние после изменений

3 Откат к контрольной точке

3.1 Процедура восстановления

Перед остановкой сервера вызывается «pg_switch_wal»: текущий сегмент WAL с изменениями закрывается и отправляется в архив. Сценарий ждёт появления сегмента в архиве, после чего откат выполняется так:

1. сервер останавливается, а его каталог данных переносится в «work/pgdata.before-restore» и сохраняется для анализа;
2. на прежнее место копируется последняя базовая копия (сценарий «restore.sh», листинг 16);
3. в «postgresql.conf» копии добавляются команда получения WAL из архива «restore_command», цель восстановления «recovery_target_name» и действие по достижении цели «promote», а в каталоге создаётся файл «recovery.signal»;
4. сервер запускается, применяет WAL из архива до точки восстановления, открывает новую линию времени и начинает принимать запросы на запись.

```
#!/bin/sh
# Восстановление кластера до точки восстановления (PITR).
# usage: restore.sh BACKUP_DATA NEW_PGDATA [RESTORE_POINT]
# Без RESTORE_POINT WAL из архива применяется до конца.
set -eu
LAB_DIR=$(cd "$(dirname "$0")" && pwd)
ARCHIVE="${WAL_ARCHIVE:-$LAB_DIR/work/wal_archive}"
src=$1 dst=$2 target=${3:-}

test ! -e "$dst"
cp -Rp "$src" "$dst"
chmod 700 "$dst"
# Журнал сервера, скопированный вместе с каталогом данных.
rm -f "$dst/server.log"

{
    echo "# lab5: восстановление из архива WAL"
    echo "restore_command = 'cp \"$ARCHIVE/%f\" \"%p\"'"
    if [ -n "$target" ]; then
        echo "recovery_target_name = '$target'"
        echo "recovery_target_action = 'promote'"
    fi
}
```

```
fi
} >>"$dst/postgresql.conf"
touch "$dst/recovery.signal"
```

Листинг 16 — Восстановление из копии («restore.sh»)

Журнал сервера во время восстановления приведён в листинге 17. Сервер применил сегменты 3 и 4 из архива и остановился на точке «lab5_before_changes». Затем он выбрал новую линию времени 2, чтобы новые сегменты WAL не перезаписали в архиве сегменты исходной линии 1 с изменениями. Ошибки «ср» при поиске файлов «.history» ожидаемы: сервер проверяет, есть ли в архиве истории линий времени, а их ещё нет.

```
12:56:14 LOG: database system was interrupted; last known up
at 2026-09-19 12:56:13 MSK
cp: .../lab5/work/wal_archive/00000002.history: No such file or
directory
12:56:14 LOG: starting point-in-time recovery to
"lab5_before_changes"
12:56:15 LOG: restored log file "00000001000000000000000003"
from archive
12:56:15 LOG: redo starts at 0/3000028
12:56:15 LOG: consistent recovery state reached at 0/3000100
12:56:15 LOG: database system is ready to accept read-only
connections
12:56:15 LOG: restored log file "00000001000000000000000004"
from archive
12:56:15 LOG: recovery stopping at restore point
"lab5_before_changes", time 2026-09-19 12:56:13.960148+03
12:56:15 LOG: redo done at 0/4000028 system usage: CPU: user:
0.00 s, system: 0.00 s, elapsed: 0.02 s
cp: .../lab5/work/wal_archive/00000002.history: No such file or
directory
12:56:15 LOG: selected new timeline ID: 2
12:56:15 LOG: archive recovery complete
cp: .../lab5/work/wal_archive/00000001.history: No such file or
directory
12:56:15 LOG: database system is ready to accept connections
```

Листинг 17 — Журнал сервера при восстановлении

3.2 Проверка результата

Состояние таблиц после отката сравнено с контрольной точкой (листинг 18): число строк и MD5 всех девяти таблиц и журнала совпадают. Значит, изменения из пункта 2 отменены полностью, а данные, существовавшие до контрольной точки, не потеряны.

table	before	restored	md5
change_log	9	9	совпадает
customer	9	9	совпадает
employee	8	8	совпадает
inspection	9	9	совпадает
instrument	9	9	совпадает
instrument_type	8	8	совпадает
order_work	11	11	совпадает
payment	8	8	совпадает
restoration_order	9	9	совпадает
service	8	8	совпадает

Листинг 18 — Контрольная точка и состояние после отката

Дополнительные проверки приведены в листинге 19: сервер вышел из режима восстановления и работает на линии времени 2; добавленных заказчика и оплаты нет, новых записей в журнале ЛРЗ тоже нет.

```
in_recovery | timeline
-----+-----
f           |      2
(1 row)

rnd_customers | rnd_payments | new_log_rows
-----+-----+-----
0 |      0 |      0
(1 row)
```

Листинг 19 — Проверки после отката

Логическая копия из того же запуска задания восстановлена утилитой «pg_restore» в отдельную временную базу данных. Её состояние также совпадает с контрольной точкой (листинг 20): между резервным копированием и точкой восстановления данные не менялись.

table	before	dump	md5
change_log	9	9	совпадает
customer	9	9	совпадает
employee	8	8	совпадает
inspection	9	9	совпадает
instrument	9	9	совпадает
instrument_type	8	8	совпадает
order_work	11	11	совпадает
payment	8	8	совпадает
restoration_order	9	9	совпадает
service	8	8	совпадает

Листинг 20 — Контрольная точка и восстановленная логическая копия

4 Анализ откаченных изменений

4.1 Журнал изменений лабораторной работы № 3

Журнал «change_log» из лабораторной работы № 3 хранится в той же базе данных, что и основные таблицы. Физическое восстановление возвращает весь кластер к моменту точки восстановления, поэтому журнал откатывается вместе с данными. После отката в нём нет ни одной записи об отменённых изменениях (столбец «new_log_rows» в листинге 19). Хэш журнала совпадает с контрольной точкой (листинг 18).

Следовательно, сам по себе журнал ЛР3 не позволяет увидеть откаченные изменения в восстановленной базе. Он полезен в копии, восстановленной на более поздний момент (подраздел 4.3), либо если хранить журнал вне откатываемого кластера: в отдельной базе данных на другом сервере или во внешнем журнале. Другой вариант — выгружать журнал перед откатом.

4.2 Архив WAL и утилита pg_waldump

Откат не удаляет сегменты WAL исходной линии времени 1 из архива: восстановленный сервер пишет WAL уже в линию 2. Утилита «pg_waldump» расшифровывает записи WAL. Она запущена по архиву от сегмента с точкой восстановления до позиции, которую вернула функция «pg_switch_wal». Первые записи приведены в листинге 21: запись «RESTORE_POINT» и следующие за ней изменения строк таблиц. Отметка «FPW» означает, что в запись включён полный образ страницы: это первое изменение страницы после контрольной точки.

```
rmgr: XLOG len (rec/tot): 98/ 98, tx: 0, lsn: 0/04000028,  
prev 0/03000100, desc: RESTORE_POINT lab5_before_changes  
rmgr: Heap len (rec/tot): 65/ 1333, tx: 759, lsn: 0/04000090,  
prev 0/04000028, desc: UPDATE off 10 xmax 759 flags 0x00 ;  
new off 11 xmax 0, blkref #0: rel 1663/16384/16386 blk 0 FPW  
rmgr: Heap len (rec/tot): 54/ 3798, tx: 759, lsn: 0/04000BE8,  
prev 0/04000B80, desc: INSERT off 10 flags 0x00, blkref #0:  
rel 1663/16384/16643 blk 0 FPW
```

Листинг 21 — Записи WAL после точки восстановления

Записи менеджера «Неар» после точки восстановления сгруппированы по файлу отношения и типу операции. Номера файлов («relfilenode») сопоставлены с именами таблиц по каталогу «pg_class» восстановленной базы данных; с момента точки восстановления таблицы не перестраивались, поэтому номера совпадают. Результат приведён в листинге 22. Он точно соответствует внесённым изменениям. Две записи «HOT_UPDATE» относятся к ценам двух услуг. Восемь вставок в «change_log» — записи журнала ЛРЗ. Запись «LOCK» в «restoration_order» — блокировка строки заказа при проверке внешнего ключа, когда добавлялась оплата.

relation	op	records
change_log	INSERT	8
customer	INSERT	1
customer	UPDATE	1
inspection	DELETE	1
order_work	UPDATE	1
payment	INSERT	1
restoration_order	LOCK	1
restoration_order	UPDATE	1
service	HOT_UPDATE	2
(9 rows)		

Листинг 22 — Изменения строк в WAL по таблицам

Записи о фиксации транзакций (листинг 23) показывают две транзакции, 759 и 760, и точное время фиксации каждой.

```
tx 759 COMMIT 2026-09-19 12:56:14.048349 MSK
tx 760 COMMIT 2026-09-19 12:56:14.072337 MSK
```

Листинг 23 — Фиксации транзакций в WAL

Таким образом, по архиву WAL можно установить, какие таблицы и строки (номер страницы и позиция) менялись, какой операцией, в какой транзакции и когда она зафиксирована. Однако в WAL нет имени пользователя и текста запроса, а значения полей хранятся в двоичном виде внутри записи. Утилита «pg_waldump» версии 14 их не расшифровывает. Поэтому WAL подходит для установления факта и времени изменений, но не для просмотра значений.

4.3 Копия на момент перед откатом

Средства системы резервирования позволяют восстановить ту же базовую копию второй раз, но уже без цели восстановления: WAL линии времени 1 применяется до конца архива. Копия поднимается в каталоге «work/pitr-latest» на порту 55434. Архивирование в ней отключено, а параметр «recovery_target_timeline = 1» заставляет сервер следовать исходной линии времени, а не линии 2, созданной откатом. Эти параметры добавляет «check.sh» после вызова «restore.sh» без имени точки.

Состояние такой копии совпадает с состоянием после случайных изменений (листинг 24), то есть откаченные изменения полностью восстановлены для анализа, а рабочая база данных при этом остаётся в откаченном состоянии.

table	after	copy	md5
change_log	17	17	совпадает
customer	10	10	совпадает
employee	8	8	совпадает
inspection	8	8	совпадает
instrument	9	9	совпадает
instrument_type	8	8	совпадает
order_work	11	11	совпадает
payment	9	9	совпадает
restoration_order	9	9	совпадает
service	8	8	совпадает

Листинг 24 — Состояние после изменений и копия на конец WAL

В этой копии журнал ЛРЗ содержит все отменённые изменения. Запрос из листинга 25 выводит записи после точки восстановления и изменённые значения. Результат приведён в листинге 26: для каждого изменения видны время, роль входа («postgres» или «reception_orlova»), операция, таблица, старое и новое значения поля или ключ добавленной и удалённой строки. Номера транзакций 759 и 760 в журнале совпадают с транзакциями из WAL (листинг 23). Это связывает физические записи WAL с пользователями и значениями.

```
-- Записи журнала ЛРЗ после точки восстановления (:since).
SELECT log_id AS id, changed_at::time(0) AS at,
       tx_id AS tx, db_role,
```

```

        operation || ' ' || table_name AS change
FROM change_log WHERE log_id > :since ORDER BY log_id;
-- Значения: изменённые поля UPDATE, ключи INSERT и DELETE.
SELECT l.log_id AS id, e.key AS field,
       l.old_row -> e.key AS old, e.value #>> '{}' AS new
FROM change_log l, jsonb_each(l.new_row) e
WHERE l.log_id > :since AND l.operation = 'U'
      AND l.old_row -> e.key IS DISTINCT FROM e.value
UNION ALL
SELECT l.log_id, k.key,
       CASE l.operation WHEN 'D' THEN k.value #>> '{}' END,
       CASE l.operation WHEN 'I' THEN k.value #>> '{}' END
FROM change_log l, jsonb_each(l.row_key) k
WHERE l.log_id > :since AND l.operation IN ('I', 'D')
ORDER BY 1, 2;

```

Листинг 25 — Записи журнала ЛРЗ после точки восстановления («sql/
log_new.sql»)

id	at	tx	db_role	change
10	12:56:14	759	postgres	U customer
11	12:56:14	759	postgres	U service
12	12:56:14	759	postgres	U service
13	12:56:14	759	postgres	U order_work
14	12:56:14	759	postgres	D inspection
15	12:56:14	760	reception_orlova	I customer
16	12:56:14	760	reception_orlova	U restoration_order
17	12:56:14	760	reception_orlova	I payment

(8 rows)

id	field	old	new
10	phone	+7 900 000-00-09	+7 999 0764774
11	base_price_kopecks	500000	914486
12	base_price_kopecks	200000	353834
13	status	planned	cancelled
14	inspection_id	1	
15	customer_id		10
16	due_on	2026-09-05	2026-09-10
17	payment_id		11

(8 rows)

Листинг 26 — Откаченные изменения в журнале ЛРЗ копии

При необходимости из этой копии можно выборочно перенести нужные строки в рабочую базу данных, например утилитой «pg_dump» с параметром «-t». Так отменяются ошибочные изменения, но сохраняются правильные, внесённые после точки восстановления.

4.4 Журнал сервера

Параметр «log_statement = mod» записывает в журнал сервера текст каждого оператора изменения данных, а «log_line_prefix» добавляет время, роль и базу данных. Журнал «server.log» находится в каталоге данных, который перед откатом был перенесён, а не удалён, поэтому записи сохранились (листинг 27).

```
12:56:14 postgres@workshop LOG: statement: UPDATE customer
  SET phone = '+7 999 ' || lpad((random() *
9999999)::int::text,
                                7, '0')
  WHERE customer_id = (SELECT customer_id FROM customer
                        ORDER BY random() LIMIT 1)
  RETURNING customer_id, phone;
12:56:14 postgres@workshop LOG: statement: UPDATE service
  SET base_price_kopecks = (base_price_kopecks
                           * (1 + random()))::int
  WHERE service_id IN (SELECT service_id FROM service
                       ORDER BY random() LIMIT 2)
  RETURNING service_id, base_price_kopecks;
12:56:14 postgres@workshop LOG: statement: UPDATE order_work
SET status = 'cancelled'
  WHERE (order_id, line_no) = (
    SELECT order_id, line_no FROM order_work
    WHERE status IN ('planned', 'in_progress')
    ORDER BY random() LIMIT 1)
  RETURNING order_id, line_no, status;
12:56:14 postgres@workshop LOG: statement: DELETE FROM
inspection
  WHERE inspection_id = (SELECT inspection_id FROM inspection
                        ORDER BY random() LIMIT 1)
  RETURNING inspection_id, order_id, inspected_on;
12:56:14 reception_orlova@workshop LOG: statement: INSERT
```

```

INTO customer (full_name, phone, email)
VALUES ('Случайный Клиент ' || (random() * 100)::int,
        '+7 900 555-00-00', NULL)
RETURNING customer_id, full_name;
12:56:14 reception_orlova@workshop LOG: statement: UPDATE
restoration_order
SET due_on = due_on + (1 + random() * 30)::int
WHERE order_id = (SELECT order_id FROM restoration_order
                  WHERE status <> 'closed'
                  ORDER BY random() LIMIT 1)
RETURNING order_id, due_on;
12:56:14 reception_orlova@workshop LOG: statement: INSERT
INTO payment (order_id, paid_on, amount_kopecks,
              method, receipt_no)
SELECT order_id, current_date,
       (1000 + random() * 9000)::int * 100, 'card',
       'RND-' || (random() * 1e6)::int
FROM restoration_order WHERE status <> 'closed'
ORDER BY random() LIMIT 1
RETURNING payment_id, order_id, amount_kopecks;

```

Листинг 27 — Операторы изменения в журнале сервера

Журнал сервера показывает, кто и когда выполнил каждый оператор, даже если транзакция была потом отменена. Но записывается только текст запроса. Для запросов с «random()» из текста нельзя узнать, какие строки и значения изменены: это видно только из журнала ЛРЗ или копии данных. Кроме того, журнал операторов может содержать секреты. Команды «CREATE ROLE ... PASSWORD» и вставка с ключом шифрования из лабораторной работы № 3 попали бы в него открытым текстом, поэтому на стенде протоколирование включается после сборки базы данных. Если хранить журнал внутри каталога данных, он попадает и в базовую копию: «pg_basebackup» копирует файл «server.log» (при восстановлении «restore.sh» его удаляет).

4.5 Сравнение источников

Итоги анализа приведены в таблице 3.

Таблица 3 — Источники сведений об откаченных изменениях

Источник	Кто	Значения	Сохраняется после отката
Журнал ЛРЗ в восстановленной БД	—	—	нет, откатывается вместе с данными
Архив WAL, pg_waldump	нет	нет, только таблица, операция, транзакция, время фиксации	да, сегменты прежней линии времени остаются в архиве
Копия на конец WAL, журнал ЛРЗ в ней	да	да, старые и новые	да, пока хранятся базовая копия и архив WAL
Журнал сервера (log_statement)	да	нет, только текст запроса	да, если журнал хранится вне восстанавливаемого каталога или каталог сохранён

Наиболее полную картину даёт сочетание системы резервирования и журнала ЛРЗ: копия на момент перед откатом восстанавливает и данные, и журнал с пользователями и значениями. Архив WAL и журнал сервера подтверждают эти сведения независимо от содержимого базы данных.

5 Защита резервных копий

Резервные копии содержат ту же информацию, что и база данных, поэтому требуют такой же защиты:

- в копиях есть персональные данные заказчиков и журнал ЛРЗ со старыми и новыми значениями строк. Каталоги копий создаются с правами 0700, файлы — 0600 (листинг 7);
- секреты таблицы «access_secret» попадают в копии только в виде шифртекста: ключ выводится из индивидуального пароля и в базе данных не хранится, поэтому копия не раскрывает секреты без этого пароля;
- базовая копия содержит системный каталог кластера, а «globals.sql» — роли с хэшами паролей SCRAM-SHA-256. По хэшам возможен перебор паролей, поэтому доступ к копиям должен быть не шире доступа к серверу;
- целостность базовой копии проверяется по манифесту, а пригодность копий — фактическим восстановлением. В этой работе обе копии, физическая и логическая, восстановлены и сверены с контрольной точкой;
- архив WAL так же важен, как базовые копии: без него откат к точке восстановления невозможен, а с ним можно восстановить любое состояние, включая удалённые данные.

Ограничения стенда: копии и архив хранятся на том же диске, что и кластер, и не шифруются. Для реальной системы копии нужно передавать во внешнее хранилище, шифровать до передачи и хранить ключ отдельно от копий. На стенде это не реализовано.

ЗАКЛЮЧЕНИЕ

Для базы данных мастерской выбрана и реализована стратегия резервирования: ежедневная физическая базовая копия с проверкой по манифесту, непрерывное архивирование WAL с интервалом не более 5 минут и ежедневная логическая копия; хранятся 7 последних копий, архив WAL очищается автоматически. Задание резервного копирования оформлено сценарием и строкой расписания crontab. На стенде оно запускалось вручную, ротация копий проверена.

После резервного копирования создана точка восстановления, затем от имени двух ролей внесены случайные изменения в шесть таблиц. Откат выполнен восстановлением базовой копии и применением WAL до точки восстановления. Число строк и хэши содержимого всех таблиц и журнала после отката совпали с контрольной точкой. Логическая копия также восстановлена и совпала с ней.

Анализ показал, что журнал ЛРЗ откатывается вместе с базой данных. Откаченные изменения всё же можно исследовать. По архиву WAL утилита «pg_waldump» показывает таблицы, операции, транзакции и время фиксации. Повторное восстановление копии до конца WAL исходной линии времени возвращает данные и журнал ЛРЗ с пользователями и значениями. Журнал сервера хранит тексты операторов. Чтобы журнал аудита переживал откат, его следует хранить вне восстанавливаемого кластера.